

Guía de Tipado Fuerte con AVFenix Types

Uso exclusivo del Motor de Esquemas, Validación Estática y Diagnósticos en JavaScript sin adoptar la capa de UI

1. Introducción: Tipado Fuerte sin Sobrecarga de Frameworks

AVFenix Types ofrece un sistema declarativo de tipado estático y comprobación de contratos de datos diseñado para proyectos JavaScript y Node.js. Aunque el compilador **AFX** soporta la generación de componentes reactivos e interfaces de usuario para GeneralJS, su motor de esquemas (*Type Engine*) puede utilizarse de forma **completamente independiente**. Esta arquitectura permite agregar tipado fuerte estático, validación de reglas de negocio y generación de manifiestos CMS a aplicaciones existentes (Express, Fastify, NestJS o scripts puros) sin modificar la capa de presentación ni incorporar dependencias de UI.

Ventaja Clave: A diferencia de TypeScript donde los tipos son totalmente borrados en compilación (*Type Erasure*), AVFenix Types mantiene **Persistencia de Esquema Dual**. El Type-Checker estático valida el código en desarrollo, mientras que los esquemas declarados se emiten en un manifiesto estructurado `.schema.json` útil para validaciones en runtime, orquestación de bases de datos y APIs.

2. Declaración de Esquemas Tipados (schema)

Para agregar tipado fuerte a un dominio de datos, se crean archivos con extensión `.avf` que contienen exclusivamente bloques `schema` o `export schema`. Cada entidad define sus atributos especificando tipos primitivos y decoradores analizados por el Type-Checker estático.

```
// models/User.avf
export schema User {
  id: string;
  nombre: string @ui(widget: "text-input", label: "Nombre Completo", required: true);
  email: string @validate(type: "email", required: true);
  edad: number @validate(min: 18, max: 120);
  activo: boolean @ui(widget: "toggle", default: true);
  fechaRegistro: date;
}

export schema Post {
  id: string;
  titulo: string @ui(widget: "text-input");
  cuerpo: text @ui(widget: "rich-editor");
  vistas: number @validate(min: 0);
  autor: User @link(relation: "many-to-one");
}
```

Resumen de Tipos Primitivos y Decoradores de Validación

Categoría	Sintaxis / Tipo	Descripción y Reglas de Validación
-----------	-----------------	------------------------------------

Tipos Primitivos	string, number, boolean	Validación básica de tipos de datos escalares.
Texto Extendido	text, date, array, object	Tipos complejos para contenidos largos, fechas y estructuras JSON.
Validación de Rango	@validate(min: X, max: Y)	Comprueba rangos numéricos. El Type-Checker valida que min <= max.
Validaciones Especiales	@validate(type: "email")	Verifica patrones estándar de correo electrónico y requerimientos.
Relaciones de Entidad	@link(relation: "...")	Conecta campos con otras entidades (one-to-one, many-to-one, etc.).
Metadatos de UI	@ui(widget: "...", label: "...")	Define la representación del campo en paneles CMS u ORMs.

3. Flujo de Comprobación Estática (Type-Checker)

El compilador AFXC incluye un **Analizador Semántico (Static Type Checker)** que inspecciona el código sin necesidad de emitir archivos ejecutables o bundles de UI. Para integrar este chequeo en el pipeline de desarrollo:

```
# Crear archivo de configuración por defecto afxc.config.json
npx afxc init

# Ejecutar SÓLO la comprobación estática de tipos en la terminal
npx afxc check ./src/models/User.avf
```

Diagnósticos Estructurados: Si se detectan inconsistencias (como un tipo no declarado, una regla min > max o una relación no válida), el Type-Checker emite alertas detalladas indicando el nivel de severidad:

```
[AFXC DIAGNOSTICS] 2 problemas detectados:
[ERROR] Regla de validación inválida en 'User.edad': min (50) es mayor que max (10).
[WARNING] Campo 'categoria' en 'Post' usa el tipo desconocido 'Category'. ¿Falta declarar la entidad 'Category'?
```

4. Reutilización de Tipos entre Módulos (import / export)

En arquitecturas grandes, los esquemas se organizan en múltiples archivos. AFXC resuelve automáticamente el **Grafo de Dependencias** respetando las sentencias de importación y soportando alias.

```
// models/Order.avf
import { User as UsuarioModel } from "./User.avf";

export schema Order {
  id: string;
  montoTotal: number @validate(min: 0);
  cliente: UsuarioModel @link(relation: "many-to-one");
}
```

5. Integración Programática en Backend (Node.js / Express)

Puedes invocar el compilador directamente dentro de tus scripts de servidor o pruebas automáticas para cargar los esquemas tipados y utilizarlos como validadores de peticiones HTTP o generadores de esquemas de BD:

```
const { AFXC } = require('afx');
const path = require('path');

// Instanciar el compilador en modo solo comprobación de esquemas
const compiler = new AFXC({ strictMode: true });

// Compilar el proyecto de modelos
const result = compiler.compileProject(path.join(__dirname, 'models/Order.avf'));

if (!result.cmsManifest.typeCheck.passed) {
  console.error("Errores de tipado detectados:", result.diagnostics);
  process.exit(1);
}

// Extraer el manifiesto JSON de entidades tipadas
const entities = result.cmsManifest.entities;
console.log("Esquemas cargados exitosamente:", entities.map(e => e.entity));
```

Resumen de Uso Exclusivo de Tipado:

1. Define tus modelos de datos en archivos .avf utilizando export schema.
2. Ejecuta `npx afxc check` en tu pipeline de CI/CD para garantizar contratos estrictos.
3. Importa el manifiesto .schema.json resultante en tu backend para configurar ORMs, APIs o validaciones en tiempo de ejecución.